

Laboratorio di Reti e Sistemi distribuiti

Progetto 4:

Server REST concorrente in C con backend SQLite

TicketFlow REST Server

Alessia Smeralda Brunello

Matricola: 544964

Indice

1	Introduzione	2
1.1	Obiettivi e funzionalità principali	2
2	Stato dell'arte	3
2.1	Architettura client-server e comunicazione TCP	3
2.2	Protocollo HTTP e modello REST	3
2.3	Server concorrenti e gestione degli eventi	3
3	Metodologia usata	4
3.1	Architettura generale del sistema	4
3.2	Gestione delle connessioni con socket non bloccanti ed epoll	4
3.3	Parsing HTTP e gestione delle connessioni persistenti	5
3.4	Implementazione delle API REST	5
3.5	Persistenza dei dati con SQLite	6
3.6	Client terminale e interazione con il server	6
4	Risultati sperimentali	7
4.1	Ambiente di compilazione ed esecuzione	7
4.2	Output Client interattivo	8
4.3	Output Server	8
4.4	Test di carico e comportamento concorrente	9
5	Conclusioni	10
5.1	Possibili estensioni	10

1 Introduzione

Il progetto realizzato consiste nello sviluppo di un server REST concorrente in linguaggio C per la gestione di `ticket`.

Il progetto prende il nome di *TicketFlow REST Server* e permette di gestire ticket associati a segnalazioni o richieste di intervento.

Ogni ticket contiene informazioni come titolo, descrizione, posizione, categoria, priorità, stato e data di creazione. L'applicazione integra più aspetti tipici dei sistemi distribuiti:

- comunicazione client-server;
- gestione concorrente delle connessioni;
- parsing di richieste HTTP;
- organizzazione REST delle risorse e persistenza dei dati tramite database SQLite.

Una parte importante del progetto è stata la scelta di non delegare la gestione HTTP a librerie esterne. Il parsing delle richieste, la costruzione delle risposte e la gestione degli header principali sono stati implementati direttamente nel codice. Questa scelta ha permesso di comprendere meglio il funzionamento dei protocolli applicativi e il modo in cui questi si appoggiano al livello di trasporto TCP.

1.1 Obiettivi e funzionalità principali

L'obiettivo principale è stato quello di realizzare un'applicazione di rete completa, capace di ricevere richieste da client differenti, interpretarle secondo una logica HTTP e restituire risposte strutturate in formato JSON.

Il sistema è in grado di:

- accettare connessioni TCP da più client;
- gestire richieste HTTP secondo uno stile REST;
- eseguire operazioni CRUD sui ticket;
- mantenere i dati in maniera persistente tramite SQLite;
- supportare connessioni persistenti tramite `keep-alive`;
- gestire errori, richieste incomplete e casi limite;
- fornire sia un client terminale interattivo sia script di test automatici.

Le funzionalità principali riguardano la creazione, visualizzazione, aggiornamento ed eliminazione di ticket. A queste si aggiunge un endpoint di controllo, `/health`, utile per verificare rapidamente che il server sia attivo e raggiungibile.

2 Stato dell'arte

2.1 Architettura client-server e comunicazione TCP

Il progetto si basa su un'architettura client-server, uno dei modelli più diffusi nei sistemi distribuiti. In questo modello il server offre un insieme di servizi e rimane in ascolto su una porta nota, mentre i client si collegano al server per inviare richieste e ricevere risposte.

La comunicazione avviene tramite socket TCP, un protocollo orientato alla connessione che garantisce la consegna ordinata e affidabile dei dati. Questa caratteristica è particolarmente utile in un'applicazione come TicketFlow, in cui le richieste HTTP e i relativi body JSON devono essere ricevuti in modo corretto prima di poter essere interpretati.

L'utilizzo di TCP comporta anche la necessità di gestire il ciclo di vita delle connessioni: apertura, ricezione dei dati, invio della risposta ed eventuale chiusura. Nel progetto questa gestione è stata estesa per supportare anche connessioni persistenti, in modo che lo stesso client possa inviare più richieste sulla stessa connessione.

2.2 Protocollo HTTP e modello REST

HTTP è il protocollo applicativo comunemente utilizzato per la comunicazione tra client e server web. Nel progetto è stato utilizzato come formato logico dei messaggi scambiati, implementando la lettura della request line, degli header principali e dell'eventuale corpo della richiesta.

Il modello REST organizza le operazioni attorno al concetto di risorsa. In TicketFlow la risorsa principale è il ticket, rappresentata dagli endpoint `/tickets` e `/tickets/id`.

I metodi HTTP vengono usati per distinguere le operazioni:

- GET per leggere risorse;
- POST per creare un nuovo ticket;
- PUT per aggiornare un ticket esistente;
- DELETE per eliminare un ticket.

Questa organizzazione rende il servizio più chiaro, perché l'operazione da eseguire non è codificata in comandi arbitrari, ma deriva dalla combinazione tra metodo HTTP e percorso della risorsa.

2.3 Server concorrenti e gestione degli eventi

Un server di rete deve poter gestire più client contemporaneamente. Una soluzione tradizionale consiste nel creare un processo o un thread per ogni client, ma questo approccio può diventare costoso quando il numero di connessioni aumenta.

Nel progetto è stato adottato un modello event-driven basato su socket non bloccanti ed `epoll`. In questo modo il server non rimane bloccato su un singolo client, ma controlla gli eventi disponibili sulle varie socket e interviene solo quando una connessione è pronta per essere letta o scritta. Questa scelta permette di gestire più sessioni con un unico event loop, rendendo il server più efficiente e più adatto a connessioni persistenti.

3 Metodologia usata

3.1 Architettura generale del sistema

Il sistema è stato organizzato in moduli separati, ognuno con responsabilità precise. Questa scelta rende il codice più leggibile e facilita sia la manutenzione sia l'estensione futura del progetto.

Modulo	Responsabilità principale
<code>main.c</code>	Inizializza il database e avvia il server sulla porta scelta.
<code>server.c</code>	Gestisce socket TCP, connessioni client, event loop, <code>epoll</code> , timeout e invio delle risposte.
<code>http.c</code>	Effettua il parsing delle richieste HTTP e costruisce le risposte HTTP complete.
<code>router.c</code>	Associa metodo e path alla corretta operazione REST.
<code>db.c</code>	Gestisce la persistenza dei ticket tramite SQLite.
<code>utils.c</code>	Contiene funzioni di supporto per l'estrazione degli ID dai path e dei valori dal JSON.
<code>client/client.c</code>	Implementa il client terminale interattivo.
<code>tests/</code>	Contiene gli script per i test funzionali e di carico.

Tabella 1: Organizzazione modulare del progetto

La struttura logica del sistema può essere rappresentata come segue:



Figura 1: Architettura generale del sistema

3.2 Gestione delle connessioni con socket non bloccanti ed `epoll`

Il server viene avviato creando una socket TCP, associandola a una porta tramite `bind` e mettendola in ascolto tramite `listen`. La socket del server viene configurata come non bloccante, così come le socket dei client accettati.

La gestione delle connessioni avviene tramite `epoll`. La socket server viene registrata nell'istanza `epoll` per intercettare nuove connessioni, mentre ogni client accettato viene rappresentato da una

struttura di sessione. Ogni sessione contiene il file descriptor della socket, l'indirizzo IP, la porta, il buffer della richiesta, il buffer della risposta e alcune informazioni di controllo.

Il server monitora eventi come:

- `EPOLLIN`, quando ci sono dati da leggere;
- `EPOLLOUT`, quando è possibile continuare l'invio di una risposta;
- `EPOLLERR`, in caso di errore sulla socket;
- `EPOLLHUP` ed `EPOLLRDHUP`, quando il client chiude la connessione.

Questa scelta evita di creare un thread dedicato per ogni client. Il server può quindi gestire più connessioni attive attraverso un unico ciclo principale. Inoltre, il limite massimo di sessioni contemporanee è controllato tramite `MAX_CLIENT_SESSIONS`; se tale limite viene raggiunto, il client riceve una risposta HTTP 503 `Service Unavailable`.

3.3 Parsing HTTP e gestione delle connessioni persistenti

La funzione principale estrae dalla richiesta il metodo, il path, la versione HTTP e l'eventuale body.

Un aspetto importante è la gestione dell'header `Content-Length`, necessario per stabilire quando una richiesta è stata ricevuta completamente.

Il server non assume che una richiesta arrivi sempre in un'unica `recv`. Per questo ogni sessione mantiene un buffer interno. La funzione che individua una richiesta completa controlla prima la presenza della sequenza `\r\n\r\n`, che separa gli header dal body, e poi verifica se il numero di byte ricevuti corrisponde al valore indicato da `Content-Length`.

Questa soluzione permette di gestire tre casi differenti:

1. richiesta arrivata completamente;
2. richiesta arrivata solo in parte;
3. più richieste presenti nello stesso buffer.

Il progetto supporta anche connessioni persistenti. Per le richieste HTTP/1.1 il server mantiene la connessione aperta di default, salvo presenza dell'header `Connection: close`.

In questo modo, un client può inviare più richieste consecutive senza dover aprire una nuova connessione TCP ogni volta. Per evitare che sessioni inattive rimangano aperte indefinitamente, è stato introdotto un timeout di inattività pari a `SOCKET_TIMEOUT` secondi.

3.4 Implementazione delle API REST

Il routing delle richieste è gestito nel modulo `router.c`. Il router riceve una struttura `HttpRequest`, analizza metodo e path, e richiama la funzione corretta del modulo database.

Il body delle richieste `POST` e `PUT` deve contenere tutti i campi necessari per costruire un ticket. Un esempio di body valido è il seguente:

```
{
  "title": "Problema Wi-Fi",
  "description": "La rete non funziona in ufficio",
  "location": "Ufficio 2",
  "category": "network",
  "priority": "high",
  "status": "open"
}
```

Le risposte sono sempre costruite in formato HTTP e contengono un body JSON. In questo modo, il comportamento del server rimane coerente sia nei casi di successo sia nei casi di errore.

3.5 Persistenza dei dati con SQLite

Per la persistenza dei ticket è stato utilizzato SQLite. La scelta di SQLite è adatta a questo progetto perché permette di avere un database locale leggero, senza dover configurare un DBMS esterno.

All'avvio del server, il modulo `db.c` inizializza il database nel percorso `data/ticketflow.db` e crea la tabella `tickets` se non esiste già. La tabella contiene i campi principali del ticket:

- `id`, chiave primaria autoincrementale;
- `title`;
- `description`;
- `location`;
- `category`;
- `priority`;
- `status`;
- `created_at`.

Le operazioni sul database utilizzano query preparate tramite `sqlite3_prepare_v2` e il binding dei parametri. Questo approccio è più sicuro e ordinato rispetto alla costruzione manuale di stringhe SQL, perché separa la struttura della query dai valori forniti dall'utente.

Viene utilizzato inoltre un mutex, `pthread_mutex_t`, per proteggere l'accesso al database.

Anche se l'implementazione del server è basata su un event loop e non su un thread per client, questa scelta lo rende più robusto e più facilmente estendibile in futuro verso un modello multi-thread.

3.6 Client terminale e interazione con il server

Oltre al server, è stato implementato un client terminale interattivo. Il client permette all'utente di eseguire le principali operazioni sui ticket tramite un menu testuale, senza dover essere obbligato a scrivere manualmente richieste `curl`.

Il client apre una sessione TCP verso il server e invia richieste HTTP/1.1 mantenendo la connessione attiva. Per le operazioni di creazione e aggiornamento, il client raccoglie i dati inseriti

dall'utente, costruisce il body JSON e lo invia al server. Le risposte vengono lette, separate dagli header HTTP e mostrate a schermo in modo più leggibile.

Questa componente è utile perché consente di testare il server in una modalità più vicina all'interazione reale, mentre gli script automatici sono più adatti alla verifica sistematica delle API.

4 Risultati sperimentali

4.1 Ambiente di compilazione ed esecuzione

Il progetto è compilabile tramite `make`. Il Makefile genera sia l'eseguibile del server sia quello del client terminale. La compilazione utilizza `gcc` con i flag `-Wall` e `-Wextra`, oltre al supporto per `pthread` e al linking della libreria SQLite.

I comandi principali previsti dal Makefile sono:

```
make
make run
make client
make test
make load_test
make clean
make clean-db
```

Il database viene salvato nella directory `data`, separando così i dati generati a runtime dai file sorgente.

4.2 Output Client interattivo

```
Connessione aperta verso 127.0.0.1 : 8080

-----
TicketFlow Terminal Client
-----
[1] Visualizza tutti i ticket
[2] Visualizza i ticket per ID
[3] Crea un nuovo ticket
[4] Aggiorna ticket
[5] Elimina ticket
[6] Health check server
[0] Esci
Scegli un'opzione: 3

Creazione nuovo ticket
Titolo: problema computer
Descrizione: il computer non si accende
Luogo: ufficio1
Categoria [Hardware/Software/Network/Other]: hardware
Priorità [Low/Medium/High]: high
Stato [Open/In_progress/Closed]: open

Risposta server:
{
  "message": "Ticket created",
  "id": 1
}

Premi solo INVIO per tornare alle opzioni ...
```

4.3 Output Server

```
[DB] Database initialized: data/ticketflow.db
[START] TicketFlow REST Server
[INFO] Listening on port 8080
[INFO] Max client sessions: 256
[INFO] Session timeout: 300 seconds
[INFO] Test single request: curl http://localhost:8080/health
[INFO] Interactive client: make client

[ACCEPT] Sessione aperta da 127.0.0.1 : 46178

[REQUEST] POST /tickets
[ROUTE] Create ticket
[DB] Ticket created with id 1
[RESPONSE] 201 Created
```

4.4 Test di carico e comportamento concorrente

Per valutare il comportamento del server con più richieste, è stato predisposto lo script `tests/load_test.sh`, basato su ApacheBench.

```
[3] Test stress: 2000 richieste, 100 concorrenti
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 200 requests
Completed 400 requests
Completed 600 requests
Completed 800 requests
Completed 1000 requests
Completed 1200 requests
Completed 1400 requests
Completed 1600 requests
Completed 1800 requests
Completed 2000 requests
Finished 2000 requests


Server Software:
Server Hostname:      localhost
Server Port:          8080

Document Path:        /tickets
Document Length:       3 bytes

Concurrency Level:     100
Time taken for tests:   0.206 seconds
Complete requests:      2000
Failed requests:         0
Keep-Alive requests:    2000
Total transferred:      194000 bytes
HTML transferred:       6000 bytes
Requests per second:    9718.08 [#/sec] (mean)
Time per request:       10.290 [ms] (mean)
Time per request:       0.103 [ms] (mean, across all concurrent requests)
Transfer rate:          920.56 [Kbytes/sec] received
```

Lo script esegue test con diversi livelli di concorrenza, aumentando progressivamente il numero di richieste e di client concorrenti.

Durante questi test, le metriche più rilevanti sono:

- il numero di richieste completate,
- eventuali richieste fallite;
- il tempo medio per richiesta;
- il numero di richieste al secondo.

L'obiettivo è che il server continui a rispondere anche con più client contemporanei, senza bloccarsi su una singola connessione.

5 Conclusioni

Il progetto ha permesso di realizzare un server REST concorrente in C, integrando diversi concetti fondamentali dei sistemi di rete e distribuiti. Il sistema gestisce comunicazione TCP, richieste HTTP, API REST, persistenza su SQLite e connessioni persistenti.

Dal punto di vista progettuale, una delle scelte più importanti è stata l'adozione di un modello event-driven basato su `epoll`.

Questo ha permesso di evitare un approccio bloccante e di gestire più client in modo efficiente.

Inoltre, l'organizzazione modulare del codice rende più semplice individuare le responsabilità dei vari componenti.

5.1 Possibili estensioni

Nonostante il sistema soddisfi gli obiettivi principali del progetto, sono possibili alcune estensioni future:

- Il parsing JSON: l'integrazione di una libreria dedicata per gestire JSON più complessi, valori annidati o caratteri di escape avanzati;
- l'introduzione di autenticazione e autorizzazione per distinguere utenti diversi e limitare le operazioni.

Ulteriori sviluppi potrebbero includere:

- paginazione e filtri nella lista dei ticket;
- logging più strutturato delle richieste;
- supporto HTTPS tramite TLS;
- esportazione dei dati o interfaccia web grafica;

In conclusione, TicketFlow REST Server rappresenta una buona base per comprendere concretamente come costruire un servizio REST a partire dai socket TCP. Il progetto mette in evidenza non solo la comunicazione tra client e server, ma anche le problematiche pratiche legate a concorrenza, buffer, richieste incomplete, persistenza e gestione degli errori.